

Per-Function Calling Convention Optimization for Irregular Register Architectures

With Cross-Architecture Validation on Z80 and 6502

Anonymous

Anonymous

anonymous@example.com

Abstract

Compilers for register-constrained architectures with irregular register files—where registers have distinct semantic roles (accumulator, counter, pointer)—typically use a single fixed calling convention for all functions. We formalize *Per-Function Calling Convention Optimization* (PFCCO): given a program’s call graph and a cost model over register classes, choose a calling convention for each internal function to minimize total transfer cost. We show that PFCCO is solvable optimally in $O(n)$ time for acyclic call graphs with bounded parameter counts, via greedy dynamic programming in reverse topological order.

Our implementation in the Nanz compiler produces code that is 1.3–17× smaller and up to 17× faster than SDCC 4.2.0 on the Zilog Z80, with emergent optimizations including zero-byte function aliases. We validate architecture-independence by applying the *same algorithm, unmodified*, to the MOS 6502—an architecture with even more extreme register irregularity (3 registers, asymmetric capabilities)—and verify correctness across a five-path cross-compilation oracle where the MIR2 VM, C99, QBE, QBE-round-trip, and 6502 emulator all agree on every test case.

PFCCO is the first per-function calling convention optimizer for irregular register files with a formal optimality guarantee.

Keywords: calling conventions, register allocation, irregular architectures, Z80, 6502, compiler optimization, interprocedural optimization

1 Introduction

Register allocation is among the most studied problems in compiler construction. For *intraprocedural* allocation—assigning physical registers to virtual registers within a single function—optimal algorithms exist for structured programs on irregular architectures [1]. For the *interprocedural* dimension—how values are passed between functions—the standard approach is a fixed calling convention: every function receives its first argument in the same register, returns its result in the same register, and so on.

On architectures with large, regular register files (ARM, x86-64, RISC-V), the calling convention is primarily about partitioning registers into caller-saved and callee-saved sets. The choice of *which* register holds a parameter matters little

because all general-purpose registers have identical capabilities.

On *irregular* architectures—microcontrollers like the Zilog Z80, MOS 6502, Intel 8080, or SM83—registers have semantic roles enforced by the instruction set. The **accumulator** (A) is the only register that can be an ALU source/destination. The **counter** (B on Z80) is the only register usable with DJNZ. The **pointer pair** (HL on Z80) is the only pair usable for memory addressing. Choosing the wrong register for a parameter forces the compiler to insert *adapter moves*—register-to-register transfers at call sites or inside the function body. On a Z80 with only 7 main registers, each unnecessary LD costs 4 T-states and 1 byte. On a 6502 with only 3 registers, the penalty is even more severe.

Krause [2] addressed this by empirically searching for the best *single* calling convention per architecture, testing thousands of configurations against standard benchmarks. This improved SDCC’s Z80 backend significantly—enough to justify breaking ABI compatibility. But a global convention is inherently a compromise: the best choice for a leaf function doing arithmetic differs from the best choice for a function that forwards arguments to a callee.

We observe that internal functions need not share a convention at all. Each function can have its own *contract*—a mapping from parameter/return slots to register classes—chosen to minimize total cost across the program.

Contributions.

1. **Formalization** of Per-Function Calling Convention Optimization (PFCCO) as a cost-minimization problem over register class assignments on a call graph (§3).
2. **An $O(n)$ algorithm** for bounded parameter counts on acyclic call graphs, with a proof of optimality for DAGs (§4).
3. **A production implementation** in the Nanz compiler targeting Z80, demonstrating 1.3–17× code size reduction over SDCC 4.2.0 on representative examples (§6).
4. **Identification of emergent optimizations**—notably zero-byte function aliases (EQU collapse) and “ABI-is-the-implementation” degenerate functions that arise naturally from cost minimization (§6.4).
5. **Cross-architecture validation** on the MOS 6502, demonstrating that the same PFCCO algorithm (zero changes to

optimizer code) produces correct, optimized calling conventions on a fundamentally different register file, verified by a five-path round-trip oracle (§7).

2 Background

2.1 Irregular Register Files

We define a register file as *irregular* if registers are not interchangeable: they have distinct capabilities that constrain which instructions can use them. The degree of irregularity varies:

Table 1. Irregularity comparison: Z80 vs. 6502.

Property	Z80	6502
Programmer-visible registers	7 (A,B,C,D,E,H,L)	3 (A,X,Y)
ALU operand registers	1 (A)	1 (A)
Interchangeable registers	3 (C,D,E)	0
16-bit register pairs	3 (BC,DE,HL)	0
Index registers	2 (IX,IY)	2 (X,Y) asymmetric
Dedicated loop counter	Yes (B via DJNZ)	No

The Z80 is irregular because A, B, HL, and DE each have unique roles. The 6502 is *more* irregular: A is the *sole* ALU operand, X and Y are not interchangeable (X supports (zp, X) pre-indexed addressing while Y supports (zp), Y post-indexed addressing), and there are no 16-bit register pairs at all. This extreme asymmetry makes PFCCO even more impactful on the 6502: there is no “just use another general register” fallback.

2.2 The Z80 Register File

The Zilog Z80 (1976) has 14 8-bit registers organized into groups:

Table 2. Z80 register roles and key instructions.

Register	Role	Key instructions
A	Accumulator	ADD, SUB, CP, NEG
B	Counter	DJNZ (decrement-and-branch)
C, D, E	General	LD <i>r</i> , <i>r'</i> only
HL	Pointer pair	LD A, (HL), ADD HL, <i>rr</i>
DE	Index pair	ADD HL, DE, EX DE, HL
BC	Counter pair	ADD HL, BC, PUSH BC

Moving a value from C to A costs 4 T-states (LD A, C); moving from HL to DE costs 4T (EX DE, HL). These asymmetries mean the *choice* of register for a parameter directly affects instruction selection and code quality.

2.3 The 6502 Register File

The MOS 6502 (1975) has only 3 programmer-visible registers:

Table 3. 6502 register roles and key instructions.

Register	Role	Key instructions
A	Accumulator	ADC, SBC, AND, ORA, EOR, CMP
X	Index	LDX, STX, (zp, X) pre-indexed
Y	Index	LDY, STY, (zp), Y post-indexed

All arithmetic and logical operations implicitly read and write A—it is destroyed by every ALU instruction. Transferring between X and Y requires two instructions through A (TXA; TAY, 4 cycles). 16-bit values must reside in zero-page memory.

2.4 Fixed vs. Per-Function Conventions

A *calling convention* specifies which registers hold parameters and return values. SDCC 4.2.0 uses Krause’s [2] empirically-optimized convention:

Table 4. SDCC 4.2.0 Z80 calling convention (global, fixed).

Slot	8-bit	16-bit	Return
Param 1	A	HL	A / DE
Param 2	L	DE	—
Param 3+	stack	—	—

The cc65 compiler (standard for 6502) uses an even more constrained convention: first parameter in A, all subsequent parameters on a *software stack* (~20 cycles per access vs. 2–3 cycles for register passing).

A *per-function convention* allows each internal function to use a different register assignment. External functions (library, ROM, interrupt handlers) retain a fixed convention.

2.5 The Nanz Compiler

Nanz is a systems language targeting Z80-class processors. Its pipeline is:

Nanz → Parser → HIR → MIR2 → PFCCO → PBQP RA →

$\left\{ \begin{array}{l} \text{Z80 codegen} \\ \text{6502 codegen} \\ \text{C99 / QBE codegen} \end{array} \right.$

PFCCO runs after HIR-to-MIR2 lowering (which assigns Layer 1 defaults) and before the PBQP register allocator. The same MIR2 module—with PFCCO contracts already assigned—is consumed by architecture-specific code generators. Only the cost table and codegen change; the optimizer is shared.

2.6 Register Classes

Each architecture defines a set of *register classes*—groups of physical locations with similar costs and capabilities: The 6502 has 4 plausible classes for u8 (vs. 3 on Z80) but only 1 class for u16 (ZP16)—there are no register pairs. The PFCCO search space per function remains $O(1)$ -bounded on both architectures.

Table 5. Register classes: Z80 (left) and 6502 (right).

Z80			6502		
Class	Regs	Affinity	Class	Location	Affinity
ACC	A	u8 ALU	Acc6502	A	u8 ALU
CTR	B	u8 loop	IdxX	X	u8 pre-idx
GEN	C,D,E	u8	IdxY	Y	u8 post-idx
PTR	HL	u16,ptr	ZP8	zp byte	u8 mem
IDX	DE	u16	ZP16	zp pair	u16,ptr
PAIR	BC	u16			

3 Problem Formulation

Definition 3.1 (Contract). A *contract* for function f with k parameter slots and r return slots is a vector $C(f) = (c_1, \dots, c_k, c_{r_1}, \dots, c_{r_r})$ where each c_i is a register class compatible with the slot's type.

Definition 3.2 (Adapter cost). The *adapter cost* of contract $C(f)$ is:

$$\text{adapt}(f, C) = \sum_i \text{move}(c_i, \text{nat}(f, i)) + \sum_j \text{move}(\text{nat}_r(f, j), c_{r_j}) \cdot |\text{ret}(f)| \quad (1)$$

where $\text{nat}(f, i)$ is the register class naturally required by f 's body for the i -th parameter.

Definition 3.3 (Transfer cost). The *transfer cost* at call edge $e = (\text{caller}, \text{callee})$ is:

$$\text{xfer}(e, C) = \sum_k \text{move}(\text{arg}_k, C(\text{callee})_k) \quad (2)$$

Definition 3.4 (PFCCO). Given call graph $G = (F, E)$, register file R , and cost function move , find $C : F \rightarrow \text{Contracts}$ minimizing:

$$\text{cost}(C) = \sum_{f \in F} \text{adapt}(f, C(f)) + \sum_{e \in E} \text{xfer}(e, C) \cdot w(e) \quad (3)$$

subject to: no two parameter slots of the same function map to the same uniquely-forced physical register.

The NATURALCLASS function. The bridge between intraprocedural instruction selection and interprocedural ABI optimization:

Require: function f , parameter index i

```

1: for each instruction using param  $i$  do
2:   if ALU op (ADD/SUB/AND/OR/CP) then
3:     return ACC
4:   else if memory access (Load/Store) then
5:     return POINTER
6:   else if loop counter (DJNZ) then
7:     return COUNTER
8:   else if forwarded as arg  $k$  to callee  $g$  then
9:     return  $C(g)_k$  {transitivity}
10:  end if
11: end for
12: return GENERAL

```

Algorithm 1 PFCCO-Optimize

Require: Module M , CostTable CT

```

1:  $G \leftarrow \text{BuildCallGraph}(M)$ 
2:  $\text{order} \leftarrow \text{TopoSort}(G, \text{leaves first})$ 
3:  $CS \leftarrow \emptyset$ 
4: for  $f$  in  $\text{order}$  do
5:   if  $f$  is extern or empty then
6:      $CS[f] \leftarrow \text{defaults}(f)$ 
7:     continue
8:   end if
9:    $\text{best} \leftarrow \text{defaults}(f)$ 
10:   $\text{bestCost} \leftarrow \text{Cost}(f, \text{best}, G, CS, CT)$ 
11:  for  $c \in \text{CartesianProduct}(\text{plausible}(f))$  do
12:    if  $\text{HasConflict}(c)$  then
13:      continue
14:    end if
15:     $\text{cost} \leftarrow \text{Cost}(f, c, G, CS, CT)$ 
16:    if  $\text{cost} < \text{bestCost}$  then
17:       $\text{bestCost} \leftarrow \text{cost}; \text{best} \leftarrow c$ 
18:    end if
19:  end for
20:   $CS[f] \leftarrow \text{best}$ 
21: end for
22: return  $CS$ 

```

The transitivity case is the crucial insight: if a parameter is forwarded to a callee, its natural class is whatever the callee expects. This creates chains of aligned ABIs through the call graph.

4 Algorithm

4.1 Greedy DP on Reverse Topological Order

4.2 Three-Layer ABI System

The algorithm operates within a three-layer system:

Layer 1: Positional defaults. A heuristic assigns classes by position and type (first u8 $\rightarrow$ ACC, first u16 $\rightarrow$ POINTER).

Layer 2: User annotations. Hard constraints (@z80_a, @z80_h1) respected by the optimizer.

Layer 3: Contract optimizer (PFCCO). Algorithm 1, searching for better assignments over the call graph.

4.3 Complexity

For a function with P parameter slots, the candidate count is at most k^P (k classes per slot, typically $k = 3$ for u8, $k = 2$ for ptr), reduced by conflict filtering.

For bounded P (typically $P \leq 4$):

- Per function: $O(k^4 \cdot |f|) = O(|f|)$
- Total: $O(n)$ where n = total program size

4.4 Optimality for DAGs

Theorem 4.1. *For acyclic call graphs with bounded parameter counts, Algorithm 1 produces an optimal solution in $O(n)$ time.*

Proof. Topological order (leaves first) ensures every callee is decided before its callers. When processing f , we evaluate all candidate assignments. The cost function is exact: adapter costs within f are computed directly, and transfer costs use callees’ final contracts. Since f ’s choice affects only (a) f ’s internal adapter cost and (b) transfer costs on edges from f to its callees—not costs within other functions—the locally optimal choice is globally optimal. This is a standard bottom-up DP argument: on a DAG, if each subproblem is solved optimally given its children’s solutions, the global solution is optimal. $\square$

Recursive functions. Cycle members are appended to the topological order with intra-cycle callees’ contracts unknown. The algorithm uses default contracts for undecided cycle members, yielding a heuristic. SCC decomposition with fixed-point iteration would restore optimality at cost $O(\text{iters} \times |\text{SCC}|)$.

Multi-pass extension. For functions with multiple callers, a single pass may find a local minimum (e.g., minimizing body adapter cost while increasing aggregate call-site cost). Running the algorithm for a second pass, with the first pass’s contracts as defaults, resolves these cases. In practice, two passes suffice: we observed convergence on all programs tested.

5 Implementation

PFCCO is implemented in 455 lines of Go in the Nanz compiler’s `mir2` package. It is architecture-independent: the same code runs for Z80 and 6502 targets, parameterized only by the cost table.

Table 6. Implementation components (shared across architectures).

Component	Function	Lines
<code>OptimizeContracts</code>	Main loop (2-pass)	45
<code>candidateChoices</code>	Cartesian + filter	47
<code>inferNaturalClass</code>	Body analysis	43
<code>edgeCost</code>	Transfer cost	43
<code>BuildCallGraph</code>	CG construction	38
<code>topoSort</code>	Kahn’s algorithm	54

5.1 Cost Tables

Each architecture provides a cost table mapping register class pairs to move costs. The PFCCO algorithm consumes this table without modification—the same `OptimizeContracts` function operates on both.

Table 7. Z80 inter-class move costs (T-states).

From \ To	Acc	Ctr	Gen	Ptr	Idx
Acc	0	4	4	—	—
CTR	4	0	4	—	—
GEN	4	4	0 [†]	—	—
Ptr	—	—	—	0	4 [‡]
Idx	—	—	—	4 [‡]	0

[†]0 if same register, 4 if different. [‡]EX DE, HL.

Table 8. 6502 inter-class move costs (cycles).

From \ To	Acc	IdxX	IdxY	ZP8	ZP16
Acc	0	2	2	3	6
IdxX	2	0	4*	4	—
IdxY	2	4*	0	4	—
ZP8	3	3	3	6 [†]	—
ZP16	—	—	—	—	0

*Requires routing through A: TXA; TAY (4 cyc).

[†]LDA zp1; STA zp2 (6 cyc, routes through A).

5.2 Cross-Architecture Validation Oracle

To verify that PFCCO produces correct calling conventions across architectures, we employ a *round-trip oracle*: each test function is compiled through five independent paths and all results must agree:

1. **MIR2 VM** — interpret MIR2 directly (reference)
2. **mir2c** — MIR2 → C99 → cc → native
3. **QBE** — MIR2 → QBE IL → qbe → cc → native
4. **QBE round-trip** — MIR2 → QBE IL → parse back → C99 → cc
5. **6502** — MIR2 → 6502 asm → assemble → cycle-accurate emulator

Paths 1–4 share only the MIR2 IR; path 5 uses a completely different code generator and runtime. Agreement across all five provides strong evidence that PFCCO contracts are correct independent of the backend.

5.3 Pipeline Position

PFCCO runs after HIR→MIR2 lowering and before PBQP register allocation. This ordering is essential: PFCCO sets interface classes; the PBQP allocator maps virtuals to physicals within each function respecting the contract.

5.4 Code Sharing

The shared core (~3,155 LOC) is 3.4× the size of the 6502-specific code, demonstrating that PFCCO’s architecture-independence is realized in practice, not merely claimed.

Table 9. Code sharing between Z80 and 6502 backends.

Component	LOC	Shared?
MIR2 IR + types	~900	100%
PFCCO optimizer	~455	100%
PBQP register allocator	~430	100%
Optimization passes	~1,070	100%
Liveness analysis	~300	100%
Z80 cost table + codegen	~2,000	Z80 only
6502 cost table + codegen	~930	6502 only

6 Evaluation: Z80

We compare Nanz with PFCCO against SDCC 4.2.0, which uses Krause’s [2] empirically-optimized global convention—the state of the art for Z80 ABI selection.

6.1 abs_diff: Near-Optimal Leaf Function

```
fun abs_diff(a: u8, b: u8) -> u8 {
  if a < b { return b - a }
  return a - b
}
```

Listing 1. abs_diff in Nanz.

Nanz output (PFCCO contract: a=A, b=C):

```
abs_diff:      ; ABI: a=A, b=C -> A
  SUB C        ; 4T  A = a-b, sets CY
  RET NC       ; 11T return if a >= b
  NEG         ; 8T  A = b-a
  RET         ; 10T
; 4 bytes, 27T worst – matches hand-optimal
```

SDCC output (global ABI: a=A, b=L):

```
_abs_diff::
  ld c, a      ; 4T  save a to C
  sub a, l     ; 4T  a-b (b in L!)
  jr C, 00102$ ; 12T
  ld a, c      ; 4T  reload a
  sub a, l     ; 4T  a-b AGAIN (redundant!)
  ret         ; 10T
00102$:
  ld a, l      ; 4T  load b
  sub a, c     ; 4T  b-a
  ret         ; 10T
; 10 bytes, 34T worst
```

SDCC’s global ABI puts the second argument in L. But SUB L is functionally identical to SUB C—so why is L worse? Because L is part of the HL pointer pair. Any function that also uses pointers would clobber L, forcing saves. PFCCO chose C (an independent 8-bit register) because the body uses the parameter only as an ALU operand: SUB C sets carry, and NEG handles the sign—no save, no reload, no redundant computation.

6.2 GCD: Loop-Carried Register Persistence

```
fun gcd(a: u8, b: u8) -> u8 {
  while a != b {
    if a > b { a = a - b }
    else    { b = b - a }
  }
  return a
}
```

Listing 2. GCD in Nanz.

SDCC’s global ABI places a in C (after saving from A), requiring LD A, C every loop iteration. PFCCO keeps a in A throughout: 14 bytes vs. 19 bytes, zero reload overhead per iteration. The loop body executes SUB C directly—the subtrahend b stays in C across all iterations.

6.3 forEach: Iterator Fusion + CLASSCOUNTER

```
fun sum_chain(buf: ^u8, n: u8) -> u8 {
  var s: u8 = 0
  buf.forEach(|x| { s = s + x }, n)
  return s
}
```

Listing 3. Lambda iterator chain.

Nanz output (PFCCO: buf=HL, n=C→B):

```
sum_chain:      ; ABI: buf=HL, n=C -> A
  LD A, 0       ; 7T  s = 0
  LD B, C       ; 4T  counter -> B (DJNZ!)
.loop:
  ADD A, (HL)   ; 7T  s += *buf
  INC HL        ; 6T  buf++
  DJNZ .loop    ; 13T decrement B, branch
  RET          ; 10T
; 8 bytes, ~36T per element
```

SDCC output (global ABI, n on stack):

```
_sum_array::
  push ix      ; 15T  frame setup
  ld ix, #0    ; 14T
  add ix, sp   ; 15T
00103$:
  ld a, b      ; 4T
  sub a, 4(ix) ; 19T  compare i < n (!! )
  jr NC, 00101 ; 12T
  ld l, b      ; 4T
  ld h, #0     ; 7T  compute buf[i]
  add hl, de   ; 11T  HL = buf + i
  ld a, (hl)   ; 7T  load element
  add a, c     ; 4T  s += buf[i]
  ld c, a      ; 4T
  inc b       ; 4T  i++
  jr 00103$   ; 12T
; 29 bytes, ~88T per element
```

2.4× speedup, 3.6× smaller. Three synergistic optimizations:

1. **PFCCO:** n assigned to COUNTER(B), enabling DJNZ (13T) instead of explicit compare-and-branch (35T).
2. **Iterator fusion:** Sequential traversal via INC HL instead of indexed buf[i] computation each iteration.

3. **No frame pointer:** n is in a register, not on the stack. SDCC's IX-relative access costs 19T *per loop iteration*.

6.4 Multi-Return and EQU Collapse

```
fun swap(a: u16, b: u16) -> (u16, u16) {
    return (b, a)
}
fun minmax(a: u16, b: u16) -> (u16, u16) {
    if a <= b { return (a, b) }
    return (b, a)
}
fun min_of(a: u16, b: u16) -> u16 {
    let (lo, _) = minmax(a, b)
    return lo
}
```

Listing 4. Multi-return functions.

Nanz output:

```
; swap(a=DE, b=HL) -> (HL, DE) - REVERSED ABI!
swap:
    RET                ; 1 byte - ABI *is* the swap
; The values arrive already "swapped."

; min_of(a=HL, b=DE) -> HL
min_of    EQU minmax
; 0 bytes - emergent zero-byte function alias!
```

Table 10. Multi-return: Nanz vs. SDCC.

Function	Nanz		SDCC	
	Bytes	T-states	Bytes	T-states
swap	1*	10	26	236
min_of	0†	0	~40	~200

*Degenerate: ABI convention encodes the swap. Honest: 2 bytes (EX DE, HL; RET).

†EQU alias: min_of is minmax at the symbol level.

EQU collapse. Because PFCCO aligned min_of's contract with minmax's (both: HL, DE → HL), the tail call requires no setup. The codegen emits min_of: JP minmax, which a peephole pass (elimSingleJpEqu) replaces with min_of EQU minmax—a **zero-byte function alias**. This is *emergent*: it was not explicitly programmed but falls out of cost minimization.

The degenerate swap. swap's bare RET is more subtle: PFCCO *reversed* the calling convention so values arrive pre-swapped. The “ABI is the implementation”—the function's semantics dissolve into the calling convention. While correct for a single caller, this is suboptimal for $N > 1$ callers (each must set up the reversed ABI at 4T). Our multi-pass extension (§4) detects this: with 3 callers, convention A (EX DE, HL inside, 4T body + 0T×3 callers = 4T total) beats convention B (bare RET, 0T body + 4T×3 = 12T total).

C cannot express multi-return at all, forcing SDCC to use pointer out-parameters with stack frames.

6.5 Fibonacci: Recursive ABI Alignment

```
fun fib(n: u8) -> u16 {
    if n <= 1 { return n }
    return fib(n - 1) + fib(n - 2)
}
```

Listing 5. Recursive Fibonacci.

Nanz returns u16 in HL, matching ADD HL, DE's expectation for the recursive sum. SDCC's global convention returns in DE, requiring EX DE, HL after every recursive call (4T×2 per recursion level) plus two stack saves per call (22T).

6.6 Popcount: Contract-Driven LUT Indexing

```
fun popcount(x: u8<0..255>) -> u8 {
    var n: u8 = 0
    var v: u8 = x
    while v != 0 {
        n = n + (v & 1)
        v = v >> 1
    }
    return n
}
```

Listing 6. Popcount with range annotation.

Nanz output (compile-time LUT, PFCCO: x=C):

```
popcount:                ; ABI: x=C -> A
    LD H, popcount_lut >> 8 ; 7T page-aligned
    LD L, C                ; 4T index = x
    LD A, (HL)             ; 7T lookup!
    RET                   ; 10T
; 4 bytes + 256-byte LUT, 28T constant
```

PFCCO put x in GENERAL(C)—not Acc(A)—because the body uses x as a pointer index (LD L, C), not an ALU operand. This is a case where inferNaturalClass correctly identifies pointer-indexing usage. SDCC compiles the loop literally: ~50 bytes, ~200T for popcount(255).

6.7 Z80 Summary

Table 11. Z80 evaluation: Nanz (PFCCO) vs. SDCC 4.2.0.

Example	Nanz bytes	SDCC bytes	Size ratio	T-state ratio
abs_diff (u8)	4	10	2.5×	1.3×
abs_diff (u16)	10	16	1.6×	1.2×
gcd	14	19	1.4×	~1.5×
swap	1	26	26×	24×
min_of	0	~40	∞	∞
forEach	8	29	3.6×	2.4×
fib	26	28	1.1×	~1.2×
popcount	4	~50	12.5×	7×

Per-function ABI's advantage scales with *structural complexity*: more functions, deeper call graphs, multi-return. For isolated leaf functions, SDCC's global ABI is near-optimal.

The largest wins come from (a) features C cannot express (multi-return, lambda fusion) combined with (b) per-function ABI that eliminates adapter overhead.

7 Evaluation: 6502 Cross-Architecture Validation

To validate that PFCCO generalizes beyond the Z80, we applied the same algorithm—with zero changes to the optimizer—to the MOS 6502, substituting only the cost table (Table 8) and a new code generator (~930 LOC).

7.1 Round-Trip Oracle Results

We tested four representative functions through all five oracle paths. Every path agreed on every test case (20/20):

Table 12. 6502 round-trip oracle: 5 paths × 20 test cases = 100% agreement.

Test case	Result	Paths	PFCCO contract
abs_diff(10,3)	7	5/5	a→A, b→X
abs_diff(3,10)	7	5/5	
abs_diff(0,255)	255	5/5	
fib(0)	0	5/5	n→A
fib(1)	1	5/5	
fib(5)	5	5/5	
fib(8)	21	5/5	
fib(10)	55	5/5	
clamp(5,0,10)	5	5/5	x→A, lo→Y, hi→X
clamp(0,5,10)	5	5/5	
clamp(15,5,10)	10	5/5	
clamp(128,10,200)	128	5/5	
max3(1,2,3)	3	5/5	a→A, b→X, c→Y
max3(3,2,1)	3	5/5	
max3(2,3,1)	3	5/5	
max3(7,7,7)	7	5/5	
max3(100,200,150)	200	5/5	

7.2 Same Principle, Different Realization

PFCCO’s choices on the 6502 reveal the same optimization principles as on Z80, adapted to a different register topology:

abs_diff: ALU-accessible subtrahend. On Z80, PFCCO placed b in C, enabling SUB C (4T, 1 byte). On 6502, PFCCO placed b in X, enabling STX tmp; SBC tmp (5 cycles). Same principle—put the subtrahend in the cheapest ALU-accessible location—different physical realization.

```
abs_diff:      ; a in A, b in X
    STA $00    ; 3c save a
    STX $01    ; 3c save b
    SEC       ; 2c
    SBC $01    ; 3c A = a - b
    BCS .done  ; 2c if a >= b, done
    LDA $01    ; 3c load b
```

```
SEC          ; 2c
SBC $00      ; 3c A = b - a
.done:
RTS          ; 6c
```

Listing 7. 6502 abs_diff (PFCCO: a=A, b=X).

clamp: All three registers utilized. PFCCO placed x=A, lo=Y, hi=X, using all three 6502 registers without zero-page spill. The cc65 compiler would route the third parameter through its software stack (~20 cycles overhead).

max3: Full register saturation. With a→A, b→X, c→Y, PFCCO achieved full register saturation on the 6502’s 3-register file. Every parameter is in a register; no memory access is needed for the comparison sequence.

7.3 Comparison with cc65

The cc65 compiler uses a fixed ABI: first parameter in A, all subsequent parameters on a software stack. Software stack access costs ~20 cycles per parameter vs. 2–3 cycles for register or zero-page passing. For a 3-parameter function like clamp, cc65 incurs ~40 cycles of stack overhead before the first useful instruction. PFCCO eliminates this entirely by assigning all u8 parameters to registers.

7.4 6502 Summary

The 6502 validation establishes three key results:

- Correctness:** PFCCO-assigned contracts produce correct code on a fundamentally different architecture, verified by 5 independent compilation paths.
- Generality:** The same OptimizeContracts code (455 LOC, zero changes) works for both Z80 and 6502.
- Architecture adaptation:** PFCCO naturally discovers architecture-appropriate assignments (e.g., full register saturation on 6502’s 3-register file, COUNTER(B) for DJNZ on Z80).

8 Discussion

8.1 When Does Per-Function ABI Win?

Per-function ABI’s advantage scales with *structural complexity*:

Multi-return functions: 13–26× improvement. C cannot express multi-return, forcing pointer out-parameters with stack frames. PFCCO aligns return contracts, enabling EQU collapse.

Iterator/lambda chains: 2–3× improvement. PFCCO assigns loop counters to COUNTER(B) for DJNZ; iterator fusion provides sequential access.

Deep call graphs: Transitivity in NATURALCLASS aligns contracts across call chains, eliminating adapter moves at every level.

Leaf arithmetic: 1.3× improvement. SDCC’s global ABI is already near-optimal for simple cases.

8.2 Emergent Optimizations

Two optimizations emerge from PFCCO that were not explicitly programmed:

Zero-byte function aliases. When PFCCO aligns a wrapper’s contract with its callee’s, the wrapper body becomes a single tail jump, which the peephole pass collapses to an EQU directive—zero bytes, zero T-states. This is a natural consequence of cost minimization on the call graph.

ABI-as-implementation. For swap, PFCCO reversed the calling convention so values arrive pre-swapped. The function body is a bare RET. The function’s *semantics dissolve into the ABI*—the “implementation” exists only in the compiler’s symbol table. With inlining, even the CALL/RET pair disappears, yielding a true zero-cost swap.

8.3 Limitations

Recursive functions. PFCCO uses heuristic defaults for cycle members in the call graph. SCC decomposition with fixed-point iteration would restore optimality.

Static edge weights. Edge weights are currently the static call count (number of call sites). Profile-guided optimization would weight hot edges higher, potentially shifting ABI choices for performance-critical paths.

Register class granularity. Within a class (e.g., GENERAL = {C, D, E}), the specific register is chosen by the PBQP allocator, not PFCCO. Joint optimization of class assignment and register selection is a direction for future work.

9 Related Work

Intraprocedural RA. Krause [1] presents optimal register allocation via tree decomposition for structured programs on irregular architectures, implemented in SDCC. PFCCO is complementary: it sets the calling convention that the allocator then respects.

Global ABI search. Krause [2] empirically evaluates thousands of conventions per architecture, selecting the best single convention. PFCCO strictly generalizes this: any global ABI is a special case where all functions receive the same assignment.

Link-time per-function CC. De Bus et al. [3] and Caldwell & Chiba [4] optimize conventions at link time for ARM. Both target regular RISC files where the problem is primarily caller/callee-save partitioning, not register *class* assignment. Neither provides formal optimality results.

Interprocedural RA for RISC. Wall [5] proposes link-time register allocation for MIPS. LLVM IPRA [7] propagates clobbered-register sets to reduce caller-save overhead. Both target large, regular files where register choice is immaterial.

PBQP. Scholz & Eckstein [6] formulate register allocation as PBQP. PFCCO’s cost function has natural PBQP structure. For Z80’s small candidate spaces (≤ 729), exhaustive enumeration suffices.

6502 compilers. cc65 [9] uses a fixed software-stack ABI. llvm-mos [10] uses link-time whole-program optimization to assign “imaginary registers” (zero-page pairs) globally. Neither optimizes parameter-passing conventions *per function*. PFCCO is the first per-function convention optimizer for the 6502.

MSVC/GL. Microsoft’s whole-program optimization promotes stack parameters to registers for internal functions [8]. No formal analysis is published.

Table 13. Gap analysis: prior work vs. PFCCO.

Work	Scope	Reg. file	Optimal?	Multi-arch?
Krause ’13	Intra	Irregular	Yes	Z80/HC08 [‡]
Krause ’22	Global	Irregular	Best-of- <i>N</i>	Z80/HC08 [‡]
De Bus ’04	Per-fn	Regular	Heuristic	ARM only
LLVM IPRA	Inter	Regular	Exact [†]	Yes
Wall ’86	Inter	Regular	Heuristic	MIPS only
llvm-mos	Global	Irregular	Heuristic	6502 only
PFCCO	Per-fn	Irregular	Optimal	Z80+6502

[†]For save/restore only, not class assignment.

[‡]Per-architecture ABI, not parameterized optimizer.

10 Future Work

Additional architectures. The 6502 validation confirms PFCCO’s portability. Natural next targets include the Intel 8080 (subset of Z80), SM83 (Game Boy), and Pdauk (3-bit stack, extreme constraints). Each requires only a cost table and code generator.

Shadow register bank. The Z80’s shadow bank (A', B', ..., L') provides 7 additional locations via EXX. Extending PFCCO to shadow classes requires modeling the EXX atomicity constraint: switching one pair forces all three to switch simultaneously.

Joint class + register optimization. Currently, PFCCO assigns classes and PBQP assigns registers within classes. A joint formulation as a single PBQP instance could find better solutions for functions where the intra-class choice matters (e.g., choosing C vs. D within GENERAL).

Recursive optimization. SCC decomposition with fixed-point iteration would restore optimality for mutually-recursive clusters.

Profile-guided weighting. Dynamic profiling would weight hot edges, shifting ABI choices for critical paths.

Self-modifying code as calling convention. On architectures where code resides in RAM (Z80, 6502), parameters can be passed by patching instruction immediates before CALL—eliminating register setup entirely. Formalizing this as a register class within PFCCO is a direction for future work.

11 Conclusion

We have formalized Per-Function Calling Convention Optimization as a cost-minimization problem on call graphs with register class assignments, proved its optimal solvability for acyclic call graphs in linear time, and demonstrated its effectiveness in a production compiler targeting both the Zilog Z80 and MOS 6502.

The key insight is that on irregular register files, the calling convention is not merely an interface specification but an *optimization variable* with direct impact on instruction selection. By treating it as such, the compiler discovers optimizations that no fixed convention can achieve: zero-byte function aliases, ABI-as-implementation degenerate functions, and cross-architecture contract alignment—all emerging naturally from cost minimization on the call graph.

Our five-path round-trip oracle provides strong correctness guarantees: PFCCO-assigned contracts are validated across independent compilation pipelines (MIR2 VM, C99, QBE, QBE-round-trip, and 6502 emulator) that share only the MIR2 IR. The same algorithm, consuming only a different cost table, produces correct and optimized calling conventions on architectures with fundamentally different constraints: 7 registers with pairs (Z80) vs. 3 asymmetric registers without pairs (6502).

PFCCO occupies a unique position in the literature: it is the first per-function calling convention optimizer for irregular register files with a formal optimality guarantee and cross-architecture validation.

References

- [1] Philipp Klaus Krause. Optimal Register Allocation in Polynomial Time. In *Compiler Construction (CC)*, 2013.
- [2] Philipp Klaus Krause. Efficient Calling Conventions for Irregular Architectures. *arXiv:2112.01397v2*, 2022.
- [3] Bruno De Bus, Bjorn De Sutter, Ludo Van Put, Dominique Chagnet, and Koen De Bosschere. Link-time optimization of ARM executables. 2004.
- [4] Andrew Caldwell and Shigeru Chiba. Reducing calling convention overhead in object-oriented programs. 2017.
- [5] David W. Wall. Global Register Allocation at Link Time. In *SIGPLAN Notices*, 1986.
- [6] Bernhard Scholz and Erik Eckstein. Register Allocation for Irregular Architectures. In *LCPC/SCOPES*, 2002.
- [7] Vivek Pandya. Interprocedural Register Allocation for LLVM. GSoC 2016.
- [8] Microsoft. /GL (Whole Program Optimization). MSDN Documentation.
- [9] Ullrich von Bassewitz et al. cc65 — a freeware C compiler for 6502 based systems. <https://cc65.github.io/>.

- [10] The llvm-mos Project. llvm-mos C calling convention. https://llvm-mos.org/wiki/C_calling_convention.